Security, Compliance and Operations

Protecting data and credentials, evidencing that the controls worked, and running the platform day to day.

SESSION 6 · 1H

Data Protection: Encryption and Secrets

SESSION 7 · 1H 30M

Compliance, Audit Logging and Monitoring

SESSION 10 · 1H 30M

Platform Operations: Compute, Cost, Monitoring and Storage

SESSION 11 · 1H 30M

Service Management, Reliability and Disaster Recovery

What this session covers

01 **Encryption** — 25 min
At rest and in transit, and what customer-managed keys change.

02 **Secrets management** — 25 min
Secret scopes, Key Vault integration, and keeping credentials out of code.

03 **Discussion and Q&A** — 10 min
Where our own position on keys and scopes needs to be decided.

Encryption is a decision taken once. Secrets management is a daily practice.

By the end, you will be able to

01 Describe encryption at rest and in transit across the control plane, compute plane and storage.

02 Explain where customer-managed keys apply and what operational obligations they create.

03 Describe secret scope patterns and the integration with Azure Key Vault.

01

Encryption

What is encrypted, and who holds the key.

at rest

in transit

customer-managed keys

rotation

Two questions

AT REST

What protects the data if somebody obtains the storage underneath it?

IN TRANSIT

What protects it while it moves between the components of the platform?

Both are on by default. The work is describing them accurately, and deciding whether the default key arrangement is acceptable to us.

Three places data rests

Workspace storage

The storage account created with the workspace: notebook revisions, job output, the DBFS root.

ADLS Gen2

Where our actual data lives, encrypted by Azure Storage on write.

Managed disks

Attached to cluster virtual machines. Spark spills to local disk when a shuffle exceeds memory.

> The third is the one people forget, and it briefly holds intermediate results of queries over sensitive data.

By default, Microsoft holds the keys

What we get

Keys generated, stored, rotated and used by Azure. Transparent, with no configuration and no operational burden.

What we give up

Any ability to withhold a key, or to prove to a third party that we can.

For most data this is a defensible position. Customer-managed keys are often adopted for the appearance of control rather than a stated requirement.

Three legs in transit

Browser → control plane	TLS, over the corporate network or a front-end private endpoint.
Control plane → compute	The secure cluster connectivity relay, established outbound from the cluster. No inbound port is opened.
Compute → storage	TLS to the ADLS Gen2 endpoint, over a private endpoint in our design.

No leg of that path carries data in clear text. That is the sentence to be able to say in a review.

The complete picture



ENCRYPTED AT REST BELOW

Workspace storage
Notebooks, job output, DBFS root

Managed disks
Shuffle spill and local cache

ADLS Gen2
The governed data itself

DEFAULT Every arrow is TLS, every store is encrypted, and Microsoft holds all the keys.

What customer-managed keys change

OUR KEY VAULT

The key is created and held by us.

→

PERMISSION GRANTED

The platform is allowed to use it, not to keep it.

→

WE CAN WITHDRAW IT

Revoke the permission and the data becomes unreadable.

That capability is the only thing CMK gives you. The encryption is no stronger, and platform access control is unaffected.

What each CMK setting covers

CMK FOR MANAGED SERVICES

What the control plane holds

Notebook source and revisions, query results, Databricks-backed secrets, personal access tokens.

CMK FOR MANAGED DISKS

What the cluster holds locally

Shuffle spill and local cache on the virtual machines' attached disks.

NEITHER COVERS — ADLS Gen2 — our actual data. That is a separate setting on the storage account, and a third decision.

What holding the key obliges us to

Availability	If Key Vault is unreachable, the platform cannot decrypt and clusters fail to start.
Deletion	Delete the key, or let the soft-delete window expire, and the data is gone permanently. There is no support path.
Access	The platform's permission on the key must survive every Key Vault policy change anyone makes.
Rotation	It becomes our schedule to run, and our evidence to produce when asked.

CMK is not a feature you enable. It is an operational commitment — take it on deliberately or not at all.

Rotation does not re-encrypt

The new version is used going forward. Existing data stays readable through the key's **version history** — which is why deleting old versions destroys data.

Rotate on a schedule

Defined, documented, and evidenced.

Keep every version

Old versions are still decrypting old data.

Alert on any change

To the key or to its access policy.

> Rehearse the loss-of-key scenario before you need to know the answer.

02

Secrets management

Keeping credentials out of notebooks and cluster configurations.

scopes | Key Vault | redaction | anti-patterns

One paste, four exposures

```
password = "Wint3r-2026!"
```

- Committed to Git, and in every revision from now on.
- Visible to everyone who can read the notebook.
- Printed in the output of any cell that echoes it.
- **Impossible to find every copy afterwards.**

It happens because it works immediately. The rest of this module is about making the correct way easier.

What a secret scope is

A named collection of secrets, and the unit access is granted on. Code references a secret **by name** and never holds the value.

The notebook is safe to share

It can be committed, reviewed and read by anyone, because there is no credential in it.

Rotation needs no code change

Replace the value behind the name, and every job picks up the new one on its next run.

Two kinds of scope

DATABRICKS-BACKED

The value lives in the control plane

Created and managed entirely through the Databricks API or CLI. Nothing else to configure.

KEY VAULT-BACKED

The scope is only a pointer

Values stay in Azure Key Vault, which remains the system of record.

With Key Vault-backed scopes, our existing rotation, access policies, logging and expiry continue to apply unchanged.

Which to use

Key Vault-backed, by default

One place credentials live, one team accountable, plus expiry dates and access logs.

Databricks-backed, by exception

Reasonable for something with no consumer outside the platform — but it is a second credential store to review.

> If there is no clear reason to split the credential store, do not split it.

Referencing a secret

```
password = dbutils.secrets.get( scope = "claims-prod", key = "sql-service-account" )
```

What is in the repository

Two names. Safe to read, review and search.

What is at runtime

The value, in memory, for the life of the session only.

Beyond notebook code

Job tasks

Read secrets under the service principal the job runs as, not under whoever wrote it.

Environment variables

Set to a secret reference rather than a literal value.

Spark configuration

Properties can reference a secret the same way.

Access to the credential follows the running identity, which is the reason production jobs run as service principals.

Redaction in output

```
print(password)
```

→

```
[REDACTED]
```

The credential does not reach the cell output.

It does not reach the notebook's saved revisions either.

A genuinely useful control that saves people regularly.

What redaction does not stop

Writing it out

Written to a file or a table, it is stored in clear text.

Transforming it

Sliced, concatenated or encoded, the literal no longer matches — so it prints.

Sending it

Over a network call, redaction never sees it at all.

> Redaction catches the accident. It does not stop anyone who wants the value, and it is not a control to cite in a review.

Four things to recognise on sight

01 A credential written into a notebook cell.

02 A literal in a cluster environment variable — visible to anyone who can view the cluster configuration.

03 A credential in a repository, including in a config file somebody decided was not code.

04 One credential shared by several teams — unrotatable without coordinating everyone, and unattributable in an audit.

The fourth is the hardest to remove, because removing it requires somebody to own the replacement.

Who can reach a scope

READ

Fetch a secret's value at runtime. What a job needs and nothing more.

WRITE

Add or replace secrets in the scope.

MANAGE

Change the scope's own permissions. Platform administrators only.

One scope per environment and consuming system. A single scope holding everything makes the permission meaningless.

Four questions for us

01 Do we have a stated requirement for customer-managed keys, or an assumption that we ought to have them?

02 If we hold a key, who holds it — and who gets woken up when it becomes unavailable?

03 Do we standardise on Key Vault-backed scopes, and which team owns the vault?

04 What do we do about the credentials already sitting in notebooks today?

Where that leaves you

You can account for how data and credentials are protected on the platform, and identify the organisational obligations that come with customer-managed keys and secret scope administration.

Encryption

On everywhere by default, at rest in three places and on every leg in transit.

Keys

Customer-managed keys buy the ability to withdraw, and oblige us to never lose it.

Secrets

Key Vault-backed scopes referenced by name, granted to groups, never pasted.

Engineering: you can describe the encryption position accurately, and there is one acceptable way to handle a credential.

Assurance and service management: you know which obligations transfer to us, and what evidence they require.

Next: what the platform records, and how the deployment is assessed against a baseline.

Compliance, Audit Logging and Monitoring

What the platform records, where those records go, and how the deployment is assessed.

01 · 25 MIN
Security profile and baseline

02 · 30 MIN
Audit logging

03 · 20 MIN
Monitoring and detection

04 · 15 MIN
Posture assessment

What this session covers

01	**Compliance security profile and platform baseline** What the profile enables, and the control categories a baseline covers.	25 min
02	**Audit logging** The system table, diagnostic log export, and the trade-off between them.	30 min
03	**Monitoring and detection** Log Analytics, Sentinel, detection use cases and who responds.	20 min
04	**Security posture assessment** The Security Analysis Tool, and using findings as a standing artefact.	15 min

Written for the information security and risk participants. This is the session to come back to.

By the end, you will be able to

01 Describe the compliance security profile and the enhanced controls it enables.

02 Explain the audit logging options and the trade-off between the system table and diagnostic log export.

03 Describe how diagnostic logs are routed to Log Analytics and consumed in Microsoft Sentinel.

04 Use a security baseline to assess a deployment against recommended practice.

01

Security profile and baseline

The switch that hardens the workspace, and the checklist behind it.

compliance profile

enhanced monitoring

baseline

control mapping

The compliance security profile

A workspace setting that turns on a set of hardening measures **together** , rather than leaving them as individual choices.

Per workspace

Enabled on the workspaces that hold regulated data, not necessarily on all of them.

A posture, not a feature

The workspace behaves differently afterwards, in ways its users will notice.

Not casually reversed

Enable it deliberately, for a stated reason, recorded as a decision.

What it turns on

Hardened images	A hardened operating system build, with automatic restart so nodes are replaced on a schedule.
Restricted node access	No interactive shell path onto the compute itself.
Node-to-node encryption	Traffic inside a cluster is protected as well as traffic in and out of it.
Enhanced security monitoring	Additional agents on the node, covered on the next slide.

Warn engineers about the restart behaviour. Long-lived clusters are replaced, and anything relying on local node state will notice.

When to enable it

Enable it when

A named compliance standard is in scope for the data in that workspace, or we have committed to a control the profile implements for us.

The cost of enabling it everywhere

Images change, clusters restart, and development workspaces get slower for no reduction in risk.

> Let the data in the workspace decide, and name the standard. "It sounds safer" is not a reason.

Enhanced security monitoring

File integrity

Monitors changes to the node's own files.

Antivirus

Scanning on the node itself.

Vulnerability scanning

Reports on the image the cluster is running.

WHERE IT GOES — Into the same audit and diagnostic streams as everything else — not a separate console to watch.

The agents consume node resources. On small clusters the cost is not zero.

Six baseline categories

Identity and access

Federation, group-based entitlement, admin role custody.

Network

Injection, private connectivity, egress filtering.

Data protection

Encryption, key custody, secret scope pattern.

Governance

Catalog structure, ownership, grant and review process.

Logging and monitoring

What is recorded, where it is sent, how long it is kept.

Operations

Change control, cluster policies, incident response.

> Writing it down converts an argument into a review. Somebody can disagree with a line rather than with the platform.

Mapping controls to requirements

Our requirement	Platform control	Evidence
Access is removed when a person leaves	SCIM deprovisioning from Entra ID	Provisioning log with timestamps
Privileged activity is logged	Audit system table	Query output, on a stated date
Data is encrypted at rest	Platform or customer-managed keys	Configuration export, key policy

Nobody can write this table for us. It needs one person who understands both of the first two columns.

02

Audit logging

What is recorded, how to reach it, and how long it lasts.

system table

diagnostic export

event contents

retention

One stream, two routes

PLATFORM ACTIVITY Every sign-in, query, grant, job run and configuration change.

↓ ↓

STAYS ON PLATFORM

The audit system table

Queryable in SQL by the people who understand the platform.

LEAVES THE PLATFORM

Diagnostic log delivery

Pushed to Azure Monitor, then Log Analytics and the SIEM.

Most organisations run both. Be deliberate about which audience uses which — "where do I look" should have one answer.

The audit system table

```
system.access.audit
```

No configuration

Available once system tables are enabled for the metastore.

Queryable in SQL

Joinable to our own reference data, usable in dashboards and alerts.

Account-wide

Every workspace on the metastore, in one table.

Governed by Unity Catalog

Access to the audit trail is itself a grant, and it should be a tight one.

Asking it a question

Which principals were granted privileges in the last thirty days, and by whom?

```
SELECT event_time, user_identity.email, action_name, request_params FROM system.access.audit WHERE action_name LIKE '%Permission%' AND event_date >= current_date() - INTERVAL 30 DAYS ORDER BY event_time DESC;
```

> Ordinary SQL, so the assurance team can own and schedule it. That is the difference between an audit capability and an audit request.

Diagnostic log delivery

Workspace resource

Diagnostic settings, as on any Azure resource.

→

Selected categories

Chosen individually — and there are many.

→

A destination

Log Analytics, a storage account, or an event hub.

It is per workspace. A new workspace logs nothing until somebody configures it, so this belongs in the deployment template, not a runbook.

Categories and latency

Selecting categories

Everything produces volume and a bill. A few produces a gap you find mid-investigation. Agree the list with whoever runs the SIEM, and record it.

Latency

Minutes, not seconds. Fine for detection; no use for anything you hoped was real-time prevention.

Both of these are decisions with an owner. Neither has a default that is right for everyone.

SIEM visibility against audit content

EXPORT TO THE SIEM

One place to correlate platform activity with the rest of the estate, using existing tooling and retention. But audit content — including query text and parameters — is copied into a system with a different access model and audience.

KEEP IT ON PLATFORM

Audit content stays inside Unity Catalog's governance and access review. But it is invisible to the team that watches everything else.

Most resolve it by exporting selected categories and keeping the richest content on-platform. Make the choice explicitly and write it into the baseline.

What an event contains

WHO	The principal — a person or a service principal, named.
WHAT	The action name, from a defined vocabulary.
WHEN	The event timestamp.
WHERE	The workspace, the source IP address and the user agent.
PARAMETERS	What was asked for. Where the detail lives, and where sensitive values can appear.

Principal, action, source IP and time together build almost every detection in the next module.

Questions it answers

Who read the claims table last month?

Who granted this person access, and when?

Did anybody sign in from an unexpected location?

What did this service principal do overnight?

> The value is not satisfying an auditor once a year. It is answering Monday morning's question in ten minutes instead of three days.

Three numbers that must agree

How long we must keep it

From our own policy and our regulator. Not from the platform.

How long the platform keeps it

The system table's retention is finite. Longer obligations need a copy to storage we control.

How long the SIEM keeps it

Log Analytics retention and archive, priced separately.

If a copy job is required, it needs monitoring — a silently failed export is a retention breach nobody notices.

03

Monitoring and detection

Where the records land, what we detect, and who acts.

Log Analytics | Sentinel | detections | response

From workspace to incident queue

Workspace
Diagnostic settings

→

Log Analytics
Selected categories land here

→

Sentinel
Same workspace, not a second copy

→

Incident queue
Worked by security operations

Which workspace

The Log Analytics workspace determines who can see the data. Decide it deliberately.

Alongside Entra ID

The useful detections correlate platform activity with sign-in logs. Put them in the same place.

Detecting privilege escalation

An account or metastore admin role granted.

A membership change on a privileged group.

Ownership of a production object reassigned.

A catalog-level grant appearing.

Build this one first: low volume, high signal. Each event should match a change record, and any that does not is worth a phone call.

Detecting anomalous access

A first-ever read of a sensitive table by a long-standing holder of the privilege.

A row volume far above that principal's normal pattern.

Access at an unusual hour for that person.

Access from an unexpected location.

All four are permitted actions, so they need a baseline of normal to mean anything. This detection needs tuning — build it only for data you would genuinely investigate.

Detecting token misuse

A token created in a workspace where creation should be disabled.

A token used from an IP range that is not ours.

A token belonging to a deactivated user, still being presented.

A long-lived token used for the first time in months.

Tokens carry no conditional access, so the log is the only place misuse appears. A dormant credential waking up is almost always worth a question.

Where native alerting ends

THE PLATFORM CAN

Alert on a query result against a threshold, show a dashboard, notify on job failure. Anything expressible as a query.

THE SIEM MUST

Correlate with the rest of the estate, hold a case, manage on-call, and track time to respond.

Operational conditions in the platform, security detections in the SIEM — and nothing important in both, or neither gets maintained.

Who responds, and to what

Security operations	Privilege escalation, anomalous access, token misuse.
Platform team	Job failures, capacity, cluster policy violations.
Data owners	Whether a particular read was legitimate. Nobody else can say.

A detection with no named data owner to adjudicate it produces an alert nobody can close. Day 3 picks this up as operational process.

04

Security posture assessment

Checking the deployment against recommended practice, repeatedly.

the tool

check catalogue

standing artefact

The Security Analysis Tool

A Databricks-provided utility that inspects a workspace's **configuration** against a catalogue of recommended practices.

Configuration, not data

It reads settings. It does not look at anybody's tables.

Runs in our workspace

As a job, on our own schedule, with output we hold.

Findings, not a verdict

A severity-ranked list to work through, not a pass or a fail.

It will find things on day one. That is the normal result, not a failure.

The check catalogue as a checklist

Identity

Admin counts, token settings, federation.

Network

Public access, secure cluster connectivity, IP access lists.

Data

Encryption settings, key configuration, secret scope usage.

Compute

Policy coverage, unrestricted creation, runtime versions.

Governance

Legacy metastore usage, mount points, catalog isolation.

Logging

Whether diagnostic delivery is configured at all.

Read it as a set of questions to ask about any workspace. That is the more durable use of it.

From one-off scan to standing artefact

It runs on a schedule

So the output is a trend rather than a snapshot.

The output is retained

Previous reports are evidence that the posture was maintained.

Findings are fixed or accepted

Accepted in writing, with a reason and a named owner.

It goes to a forum

The same meeting, every time. This is what stops it becoming shelfware.

A scan run once gets fixed and forgotten. A scan run repeatedly demonstrates that the posture is maintained.

Where that leaves you

You understand what the platform records, where those records are sent, and how the deployment's security posture is assessed and evidenced on an ongoing basis.

The records

The audit system table as the primary path, diagnostic export for the SIEM.

The detections

Escalation first, then anomalous access and token misuse, with response named.

The assessment

A baseline written down, and a scheduled scan reported to a standing forum.

THREE DECISIONS THIS SESSION LEAVES WITH YOU

The retention numbers, the exported category list, and who adjudicates a data-access alert.

End of Day 2. Day 3 moves to running the platform day to day.

DAY 2 · COMPLETE

Platform Operations

Compute, cost, monitoring and storage — the day-to-day operational surface of the platform.

01 · 25 MIN
Compute operations

02 · 30 MIN
Cost governance

03 · 20 MIN
Monitoring and observability

04 · 15 MIN
Storage operations

What this session covers

01	**Compute operations** Lifecycle, autoscaling, the three kinds of compute, and serverless.	25 min
02	**Cost governance** DBUs, tag-based attribution, budgets, and where spend goes wrong.	30 min
03	**Monitoring and observability** System tables, query history, Azure Monitor, and what to alert on.	20 min
04	**Storage operations** Container layout, volumes, workspace storage, and retention.	15 min

Nothing here is decided once. All four are practices the platform team keeps up.

By the end, you will be able to

01 Describe compute lifecycle management and autoscaling behaviour.

02 Explain DBU cost governance, tag-based attribution and budget controls.

03 Use system tables and monitoring dashboards for operational and cost visibility.

04 Describe the storage administration model under Unity Catalog.

01

Compute operations

How compute comes and goes, and which kind belongs where.

lifecycle autoscaling serverless policies

The cluster lifecycle

Defined
A configuration. Costs nothing.

→

Starting
Minutes, while machines are acquired.

→

Running
The only state that costs anything.

→

Terminated
Definition kept, ready to start again.

A terminated cluster is free, so nothing should run out of convenience. Start-up takes minutes, which is exactly why people leave them running.

Autotermination

The **highest-value setting on the platform** : no activity for the configured period, and the cluster shuts itself down.

Too long

You pay for lunch breaks, meetings and overnight idling.

Too short

People lose session state mid-thought and start resenting the platform.

> Put it in the cluster policy as a mandatory maximum, not a default. The cluster that costs you most will be the one where somebody raised it.

Autoscaling is not symmetric

SCALING UP

Fast and eager

Workers are added as soon as there is pending work, within the job.

SCALING DOWN

Slow and cautious

A worker holding shuffle data that may still be needed is not discarded.

A cluster that briefly needed twenty workers does not return to two the moment the peak passes.

Autoscaling is not a cost control

It is a performance control with a **cost ceiling** . The maximum is what you have agreed to spend — and a bad query will spend it.

Set the maximum

From what the workload needs, not what the team hopes it might.

Keep the minimum low

The minimum is what you pay while a cluster idles before autotermination fires.

Pinned at maximum

That is a tuning problem, not a sizing one. Look at the query.

Three kinds, three rates

All-purpose

Interactive and shared, started and stopped by people. The higher rate.

Job compute

Created for a scheduled run, destroyed at the end. A materially lower rate.

SQL warehouse

Serves queries and dashboards, scaling for concurrency. Its own rate.

Anything scheduled runs on job compute. A nightly pipeline still on the cluster it was developed on is the most common line item worth reclaiming.

What serverless changes

What you gain

Near-instant start, no capacity to size, and no idle cost — the compute does not belong to us between uses.

What you give up

The machines sit in Databricks' subscription, not in our VNet. Day 1 covered when that is acceptable.

BILLING A single DBU rate with no separate infrastructure line — easier to attribute, harder to compare.

Serverless does not remove cost governance. It moves where you look for it.

Policies, revisited operationally

Autotermination maximum

Mandatory, and capped where nobody can raise it.

Worker ceiling

The agreed upper bound on what a workload may spend.

Instance families

An approved list, so nobody provisions memory-optimised nodes by accident.

Enforced tags

The foundation of everything in the next module.

> Get policies right before the cost work. Attribution without enforced tags is guesswork.

02

Cost governance

What is being spent, by whom, and how it is controlled.

DBUs

tags

attribution

budgets

One cluster, two charges

FROM DATABRICKS

DBUs

The software charge, measured per unit of processing per hour. The rate depends on the kind of compute.

FROM AZURE

Infrastructure

The virtual machines, managed disks and networking underneath the cluster.

A saving on one is not automatically a saving on the other. When somebody asks what a cluster costs, the honest answer has two numbers in it.

What moves the DBU number

Kind of compute	The biggest single lever, and a configuration choice rather than a tuning exercise.
Workers × time	How many ran, and for how long. Autoscaling and autotermination.
Workspace tier	Affects the rate applied to everything in it.
Runtime features	Photon and similar change the rate while usually cutting duration. Measure, do not assume.

The two biggest levers are both decisions, not optimisations.

Four tags are enough

`cost_centre`

So finance can charge it.

`team`

So somebody can be asked about it.

`environment`

So development is separable from production.

`workload`

So one expensive pipeline can be found.

Agree the allowed **values** , not just the keys. Free text produces fourteen spellings of one team.

Decide before go-live. Historical usage cannot be retagged — untagged spend stays untagged forever.

Enforcing tags through policy

A required tag in the policy means a cluster without it **cannot be created**.

Created outside a policy

Unrestricted creation bypasses the requirement entirely.

Created before the policy

Existing clusters are not retrospectively tagged.

Serverless

Attribution comes from the workspace and running identity, not a cluster tag.

Check tag coverage as a number rather than assuming it. "We enforce tags" and "all our spend is tagged" are different claims.

The billable usage table

```
system.billing.usage
```

One row per usage record — workspace, SKU, DBU quantity, period.

The tags that were attached at the time, carried on the row.

Governed by Unity Catalog, like any other table.

Queryable in SQL, so reporting can be scheduled and owned.

Cost becomes a reportable fact rather than a monthly surprise.

Attribution by team

```
SELECT custom_tags.team, sku_name, SUM(usage_quantity) AS dbus FROM system.billing.usage WHERE
usage_date >= date_trunc('MONTH', current_date() - INTERVAL 1 MONTH) GROUP BY ALL ORDER BY dbus DESC;
```

The same query every month — so a scheduled job, not a task.

The untagged bucket is the honest measure of whether tagging works.

It returns DBUs. Currency needs the rate card.

A budget alert is not a cap

What it does

Tells named recipients when consumption passes a threshold you set.

What it does not do

Stop the spending. Nothing on this platform does.

> The control is the alert plus a named person who acts. Set the threshold below the point at which you would be uncomfortable, not at it.

Two reports, two jobs

SYSTEM TABLE

Who spent it

DBUs by tag, by workload, at whatever granularity you query.

AZURE COST MANAGEMENT

How much it cost

Currency, including infrastructure, by Azure resource tag.

They will not agree line for line. Reconcile at total level monthly, and make sure both use the same tag vocabulary — or the reports cannot be joined at all.

Where the money actually goes

An all-purpose cluster left running over a weekend.

A SQL warehouse idling for one dashboard nobody opens.

A development cluster sized like production, because it was cloned from it.

A job retrying on failure all night.

A pipeline reading the whole table because the incremental logic was never finished.

Streaming left running after the proof of concept ended.

All six are findable in the usage table in five minutes — if somebody is looking. The control is a standing review with an owner, not a cleverer configuration.

03

Monitoring and observability

What the platform tells you about itself, and who hears it.

system tables | query history | Azure Monitor | alerting

The platform describing itself

`system.billing`

Consumption records.

`system.access`

The audit trail.

`system.compute`

Cluster and warehouse events.

`system.query`

Execution history.

`system.lakeflow`

Job and pipeline runs.

One namespace

Same governance and grants as any other table.

Every operational question about this platform is a SQL query. Dashboards, finance reporting and assurance evidence all come from one place.

”It is slow”

Query history records who ran it, how long it took, how much it scanned, whether it spilled, and how long it queued.

Long duration, small scan

A queueing or concurrency problem. Look at the warehouse.

Long duration, enormous scan

A query or table design problem. Look at the code.

Two different owners. Telling them apart in thirty seconds saves a lot of misdirected effort.

Watching the data, not the platform

Profiles a table

Row counts, null rates and distributions over time.

Measures drift

Against a baseline you declare.

Produces metrics tables

Queryable and alertable like anything else.

> Most "platform problems" are data problems: the job succeeded, the dashboard rendered, and the upstream feed arrived half empty. Compute monitoring will never tell you that.

Where Azure Monitor fits

Correlation

A pipeline failure and a storage throttling event look unrelated until they are in the same query.

Existing channels

Alerting through what the operations team already watches, not another notification source.

Same mechanism as session 7 — diagnostic settings to Log Analytics. Same rule too: never duplicate an alert in both places.

Four alerts, not forty

Production job failed	After retries are exhausted — not on the first transient failure.
Job missed its window	Still running rather than failed — invisible to a failure alert.
Spend past a threshold	From the budget alert, to a named recipient.
Warehouse queueing	Beyond an agreed wait. The one users feel.

If a fifth is proposed, ask who will act on it at three in the morning.

Can the recipient fix it?

Job failure	The team that owns the pipeline. The platform team cannot fix their logic.
Platform and capacity	The platform team.
Spend	The budget holder, not whoever built the cluster.
Data quality	The data owner.

> If the recipient cannot fix it, it is a report — put it in a weekly summary, not into somebody's phone at night.

04

Storage operations

What sits where, and who looks after it.

container layout

volumes

retention

Keep the layout boring

`prod-bronze`
Raw, as it arrived.

`prod-silver`
Cleaned and conformed.

`prod-gold`
Modelled for consumption.

Do not mirror the org chart. Teams reorganise; containers do not.

Do not put every environment in one storage account. Several Azure controls apply at account level.

> The layout is effectively permanent. Moving data later means rewriting every external location and table definition pointing at it.

Workspace storage is not data storage

WORKSPACE STORAGE

The platform’s own artefacts

Notebook revisions, job output, the DBFS root. Created with the workspace.

DATA STORAGE

The ADLS Gen2 accounts we designed

Governed through Unity Catalog external locations and managed storage.

Nothing in workspace storage should be business-critical — it is outside the retention regime we designed for data. And no table physically lives "in the workspace".

Every file request gets a volume

"I need to put a file somewhere the platform can read" is the most frequent operational request you will get.

The answers to refuse

A mount, a shared key, or a copy into workspace storage. All three are problems you inherit.

The answer to give

A volume: a named object in a schema, with grants, lineage and an audit trail.

There is no second answer. That is what makes it an operational standard rather than a preference.

Retention is four mechanisms

Azure lifecycle policies

Move or delete blobs by age. They know nothing about tables.

Delta history and VACUUM

History enables time travel; VACUUM is what reclaims the space.

Managed against external

Dropping a managed table removes the data. Dropping an external one does not.

The actual requirement

From records management. None of the three above know what it is.

A lifecycle rule that deletes files a Delta table still references breaks the table. Somebody has to reconcile these four deliberately.

Where that leaves you

You can describe how compute and storage are administered, how platform cost is attributed and controlled, and what operational signals the platform produces.

Compute

Job compute for anything scheduled, autotermination capped by policy.

Cost

Two charges, four enforced tags, and the usage table as the report.

Monitoring

System tables for everything, four alerts, named recipients.

Storage

A boring layout, volumes for files, retention reconciled deliberately.

THE THREE THAT DECIDE WHETHER THIS IS WELL RUN

Enforced tags from day one, a short alert list with named recipients, and a standing review of spend and posture.

None of those three are technical. All three need an owner.

SESSION 10 · COMPLETE

Service Management, Reliability and Disaster Recovery

How the platform is supported in service, and how it recovers from failure.

01 · 25 MIN
Supporting the platform in service

02 · 20 MIN
High availability and backup

03 · 30 MIN
Disaster recovery

04 · 15 MIN
Runbooks and testing

What this session covers

01 **Supporting the platform in service** — 25 min

Shared responsibility, ownership, incidents, change and escalation.

02 **High availability and backup** — 20 min

Zone redundancy, compute resilience, time travel and deep clone.

03 **Disaster recovery** — 30 min

Objectives, managed against self-built, and the patterns available.

04 **Runbooks and testing** — 15 min

What the runbook contains, and the cycle that keeps it true.

High availability is largely given to us. Disaster recovery largely is not.

By the end, you will be able to

01 Describe the support model and escalation path across the organisation, Microsoft and Databricks.

02 Distinguish high availability, backup and disaster recovery, and the failure each addresses.

03 Compare managed disaster recovery with a self-built approach and identify the dependencies of each.

04 Describe the content of a disaster recovery runbook and the testing cycle around it.

01

Supporting the platform in service

Who owns what, what breaks, and who to call.

responsibility incidents change escalation

Three parties, three boundaries

MICROSOFT

The infrastructure

The region, virtual machines, storage, and the Azure services around us.

DATABRICKS

The control plane and software

The interface, the scheduler, the runtime, and the Unity Catalog service.

US

Everything we configure and build

Identity, network, policies, grants, pipelines and data.

Almost every incident you will actually work sits in the third column. An incident spent arguing about whose problem it is has already gone badly.

Who administers what

Platform team	Workspaces, networking, cluster policies, account configuration.
Data owners	Catalogs, schemas and the grants on them.
Engineering teams	Their own pipelines, jobs and code.
Security	The baseline, the detections and the incident process.

> For any object, one of these four should be nameable in ten seconds. Otherwise it becomes the platform team's by default.

What actually breaks

A job failed.

A cluster will not start — quota, policy, or no addresses left in the subnet.

Somebody cannot access a table.

A query is slow, or a warehouse is queueing.

A pipeline succeeded and the data is wrong.

A credential expired overnight.

Five of the six are configuration or data problems, not platform faults. That ratio is why first-line response matters more than vendor support.

First-line routine

01 Identify the principal and the object — who was doing what, to what.

02 Check the obvious configuration: policy, quota, subnet capacity, credential expiry.

03 Read the relevant system table for the event, rather than asking the user to reproduce it.

04 Only then decide whether it is ours, Microsoft's or Databricks'.

Most tickets close in the first three steps. A case raised before them comes back with a question you could have answered.

The release path for changes

Infrastructure

Workspaces, networking, policies — Terraform and normal change control.

Governance

Catalogs, grants, ownership — as SQL in the repository.

Code

Git and the deployment bundle.

THE FOURTH CLASS

Databricks upgrades the runtime and platform on its own schedule — a change to our environment that we did not raise.

Track the release notes, and pin runtime versions in policies, so an upgrade is adopted rather than received.

Two support routes, one gap

AZURE SUPPORT

Infrastructure, quotas and Azure service issues, through our existing agreement.

DATABRICKS SUPPORT

Control plane, runtime and Unity Catalog, with its own severities and response times.

THE GAP A problem spanning both — a cluster failing on a networking interaction — becomes two cases, and we coordinate them.

Confirm the Databricks support tier and who may raise a case before an incident, not during one.

02

High availability and backup

What survives a failure inside the region.

availability zones

time travel

deep clone

What zone redundancy covers

Control plane

Operated across zones by Databricks. Read the service level agreement; there is nothing to configure.

Storage

Zone-redundant only if that replication option was chosen at account creation.

Compute

Not covered. A cluster runs in one zone at a time.

Check the storage replication setting rather than assuming it. It is chosen once and hard to change afterwards.

Designing compute for zone failure

A cluster in a failed zone does not migrate. The job **fails and is restarted** elsewhere.

Do not pin a zone

Let the platform choose, so a restart can land elsewhere.

Make jobs restartable

Idempotent writes and checkpoints, so a rerun is safe.

Size the subnets

A whole workload restarting needs addresses to be available.

Time travel protects against bad writes

```
RESTORE TABLE prod.claims.paid_claims TO VERSION AS OF 412;
```

The job that ran twice.

The delete with the wrong predicate.

The transformation that silently dropped a column.

> Bounded by the retention setting and by VACUUM — so not an archive. And it protects against corruption, not against losing the region.

Deep clone crosses boundaries

What it does

Copies data and metadata to another catalog, storage account or region. Incremental afterwards.

Where it is used

A point-in-time copy held elsewhere — and the data replication mechanism in self-built DR.

> A clone is a scheduled job with an owner. One that quietly stopped three months ago is worse than none, because it produces false confidence.

Three failures, three mechanisms

High availability	A component failed inside the region. Zone redundancy and a restart, in minutes.
Backup	The data is wrong or was destroyed. Time travel or a clone, to a point in time.
Disaster recovery	The region is gone. A second region that we have prepared in advance.

> The common mistake is treating zone redundancy as disaster recovery. Both zones are in the region that just failed.

03

Disaster recovery

Losing a region is the scenario the platform helps with least.

RTO and RPO

managed DR

self-built

DR as code

Two numbers first

RTO

Recovery time objective

How long the workload may be unavailable.

RPO

Recovery point objective

How much data we may lose, expressed as time.

> Business decisions, not technical ones. The first answer is always zero and zero — ask what they would pay for it and the real numbers appear.

Objectives are per workload

Regulatory reporting	A hard external deadline. The tightest objective we hold.
Claims operations	Same-day recovery.
Actuarial modelling	Days are usually tolerable — the models can be rerun.
Exploratory analysis	A rebuild is acceptable.

A platform-wide objective gets set at the most demanding workload, and we pay that for the sandbox too. Tiering is the largest cost lever in DR.

Managed disaster recovery

Replicates the **workspace** to a secondary region, so it can be brought up without us rebuilding it.

What it removes

Recreating clusters, jobs, policies, permissions and workspace objects by hand, under pressure.

What it is not

A complete answer. Our data still has to be present in the secondary region by other means.

Confirm the current scope and regional availability with Databricks before designing around it.

What it still depends on

Data

Storage replication is ours to arrange — geo-redundant storage, or clone jobs.

Identity

People must be able to authenticate there. That is Entra ID and our network, not the platform.

Network

Its own VNet, subnets, private endpoints and DNS. An unprepared network turns hours into days.

Access to invoke it

Whoever declares the failover must be able to reach the account console at the time.

> The last one is assumed far more often than it is tested. Rehearse it.

Self-built: four moving parts

Workspace assets

Notebooks, jobs, policies, dashboards — deployed from source control, not copied.

Unity Catalog metadata

Catalogs, schemas, grants and ownership — applied as SQL from the repository.

Identity

SCIM provisioning pointed at the secondary account, so the principals exist there.

Data

Deep clone or storage replication, on a schedule that matches the RPO.

Which of the four do we have today? Teams routinely build one or two and believe they are covered.

Data is the easy half

Data

A solved problem. Clone jobs or geo-redundant storage, running unattended.

Metadata

The same catalogs, schemas, grants and ownership. Missing any of it means data present and unreadable.

A governance model that exists only as two years of clicks cannot be recreated elsewhere under time pressure. That, not the data, is what makes a failover fail.

Active-passive or active-active

ACTIVE-PASSIVE

The secondary waits

Cheaper. Recovery time is however long it takes to bring it up — the number the runbook must be honest about.

ACTIVE-ACTIVE

Both regions run

Much faster recovery, roughly double the cost, and real complexity in keeping data consistent both ways.

> Active-passive with a tested runbook is the proportionate answer for most workloads. Active-active is justified by a specific objective, not a general preference.

DR as code

If the primary came from Terraform and SQL in source control, the secondary is **the same configuration with different parameters** .

No drift

Both regions come from one definition.

One change, both regions

A change to the primary is a change to the secondary.

Tested continuously

The capability is exercised every deployment, not once a year.

> If the primary was built by hand, there is no disaster recovery — only an aspiration. Nobody reproduces two years of clicks under pressure.

04

Runbooks and testing

What turns a design into a capability.

the runbook

failover and failback

testing

Six sections

01
Who declares a disaster — named roles, not a team.

02
The criteria for declaring one, decided in advance.

03
The ordered steps, specific enough to follow without improvising.

04
Who to notify and when — the business, and the regulator if relevant.

05
How to verify the secondary is actually serving correctly.

06
The criteria for failing back.

> The test is not whether the runbook is accurate. It is whether somebody who did not write it can follow it.

Failback is the hard direction

FAILOVER

Stop using the primary, bring the secondary up, verify it, redirect users. Comparatively simple.

FAILBACK

The secondary has been accepting writes. Returning them to the primary is a data merge, not a switch.

> Decide in advance whether failback is required at all, or whether the secondary simply becomes the new primary.

Three levels of test

Tabletop

Walking the runbook aloud. Cheap, quarterly. Finds the missing step and the person who has left.

Component test

One piece for real — restore from a clone, confirm identity in the secondary. Annual.

Full failover

Actually running the business in the secondary. Expensive, and the only true recovery time.

> Run the cheap one often. An untested runbook is a document, not a capability.

Four artefacts

The objectives

Agreed and signed off, per workload.

The runbook

Versioned, and dated.

The test records

What was tested, when, and what was found.

The remediation

Evidence that the findings were closed.

A test that finds five problems and closes none evidences a gap, not a capability. Take the set to the same forum as the posture report.

Where that leaves you

You understand how the platform is supported, what protects it against zone and region failure, and what the organisation must build, document and test to meet its recovery objectives.

Support

Three parties, four owners, and two vendor routes with a gap between them.

Availability

Zones cover the region's components. Our jobs fail and restart.

Recovery

Objectives per workload, metadata as code, and a prepared secondary.

Discipline

A runbook somebody else can follow, tested on a cadence, with findings closed.

THE PATTERN ACROSS THREE DAYS

The platform provides the mechanism. We provide the decision — group design, catalog structure, key custody, tags, and recovery objectives.

The platform is capable. Whether it is well run is decided by that list.

SESSION 11 · COMPLETE